Simple Linear Regression

Birth Weight Dataset / Cow Yield Dataset

Problem Statement

Implement **Simple Linear Regression** using the following methods:

1. **Regular Method**
   - Find b_0, b_1
   - Compute error y bar
   - Plot regression line
2. **Normal Equations (Least Squares Method)**
   - Using direct formulas
   - a_1 fixed
   - a_0 fixed
   - Simultaneous change of a_0, a_1
3. **Matrix Inversion Method**
   - Form matrices
   - Find determinant, adjoint, inverse
   - Solve using A^{-1}B
4. **Gradient Descent Method**
   - Update rule

$$b_j^{new} = b_j^{old} - \alpha \frac{\partial J}{\partial b_j}$$

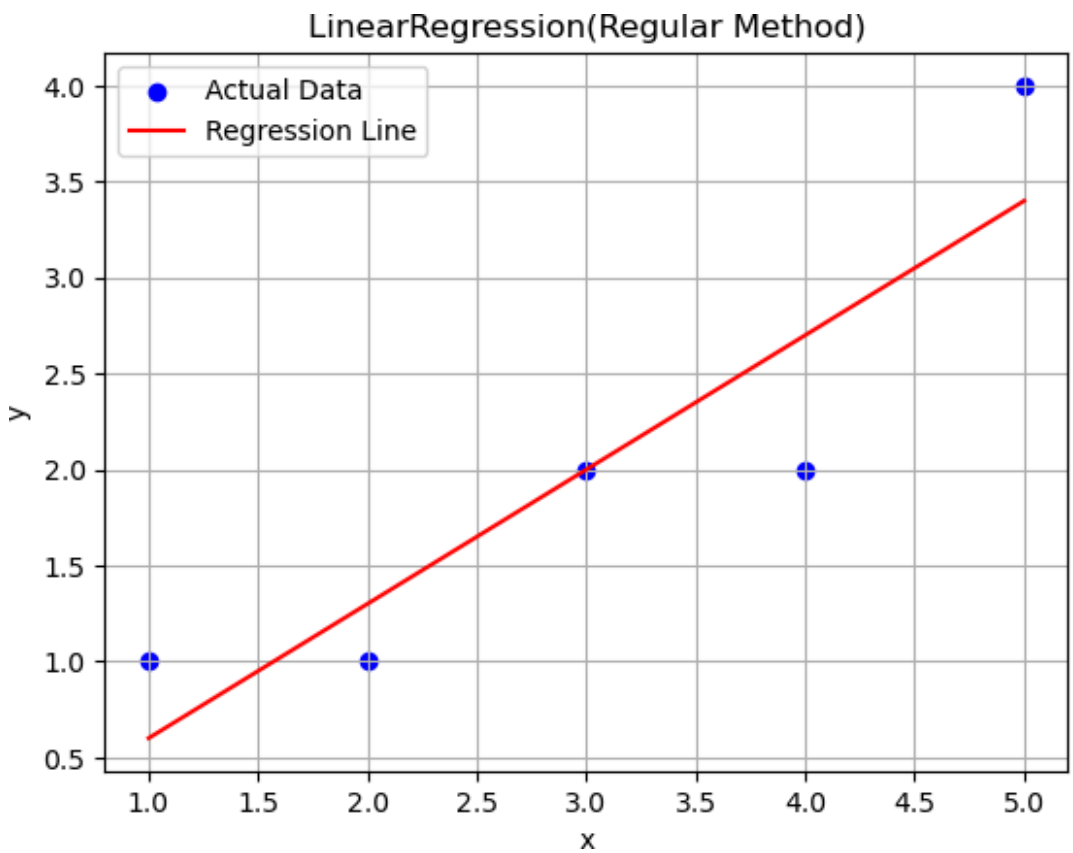
   - With stopping Criteria

```
import numpy as np
 import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

Regular Method

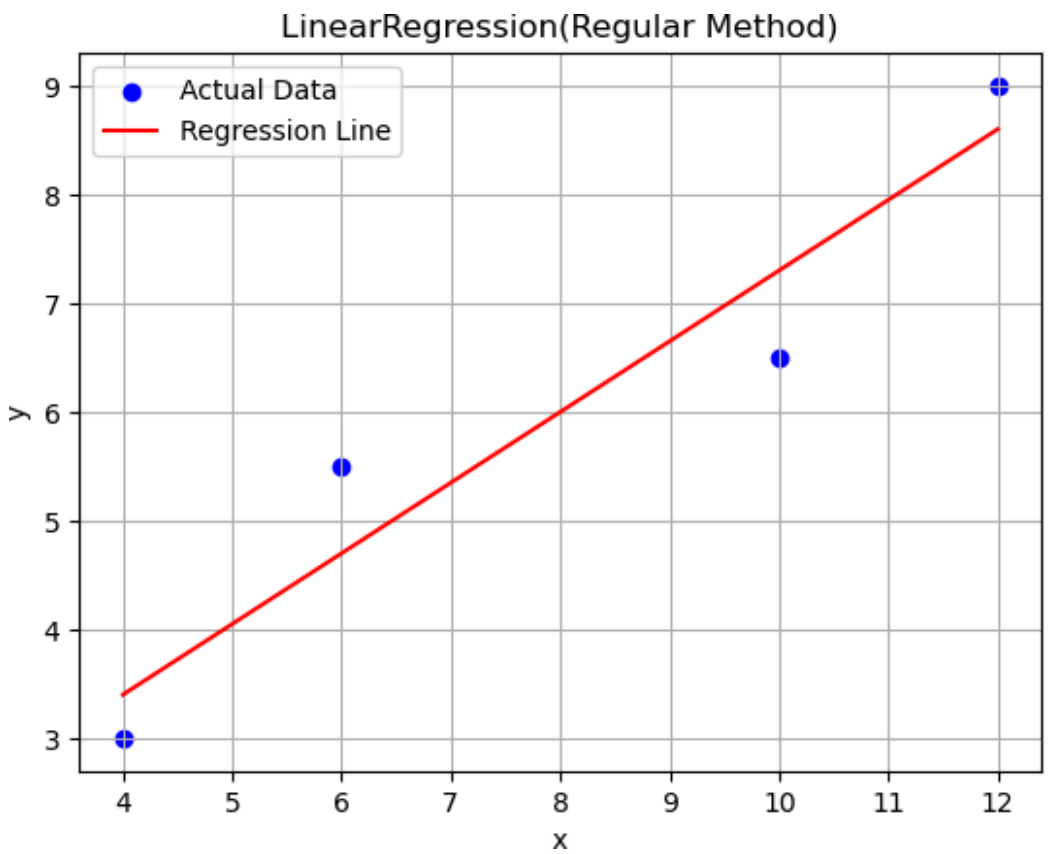
```
def regular_method(x, y): n = len(x)

    x_mean = np.mean(x)
    y_mean = np.mean(y)

    # b1 (slope)
    numerator = np.sum((x - x_mean) * (y - y_mean)) denominator = np.sum((x -
    x_mean) ** 2)
    b1 = numerator / denominator

    # b0 (intercept)
    b0 = y_mean - b1 * x_mean

    print(f"Mean of x: {x_mean}") print(f"Mean
    of y: {y_mean}")

    print(f"b0 (intercept): {b0}")
    print(f"b1 (slope): {b1}")

    y_pred = b0 + b1 * x print(f"y_pred:
    {y_pred}")

    error = y - y_pred print(f"error: {error}")

    plt.scatter(x, y, color="blue", label="Actual Data") plt.plot(x, y_pred,
    color="red", label="Regression Line") plt.xlabel("x")
    plt.ylabel("y") plt.title("LinearRegression(Regular Method)")
    plt.legend()
    plt.grid()
    plt.show()

x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4]) print(f"Birth Weight
Data:")

print(f"x:{x}\ny:{y}")
regular_method(x,y)
```

```
Birth Weight Data:
x:[1 2 3 4 5]
y:[1 1 2 2 4]
Mean of x: 3.0
Mean of y: 2.0
b0  (intercept):  -0.09999999999999964
b1 (slope): 0.7
y_pred: [0.6 1.3 2.                2.7 3.4]
error: [ 0.4 -0.3                0.     -0.7      0.6]
```




```
x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])
print(f"Cow Yield Data:")

print(f"\nx:{x}\ny:{y}\n")
 regular_method(x,y)
```

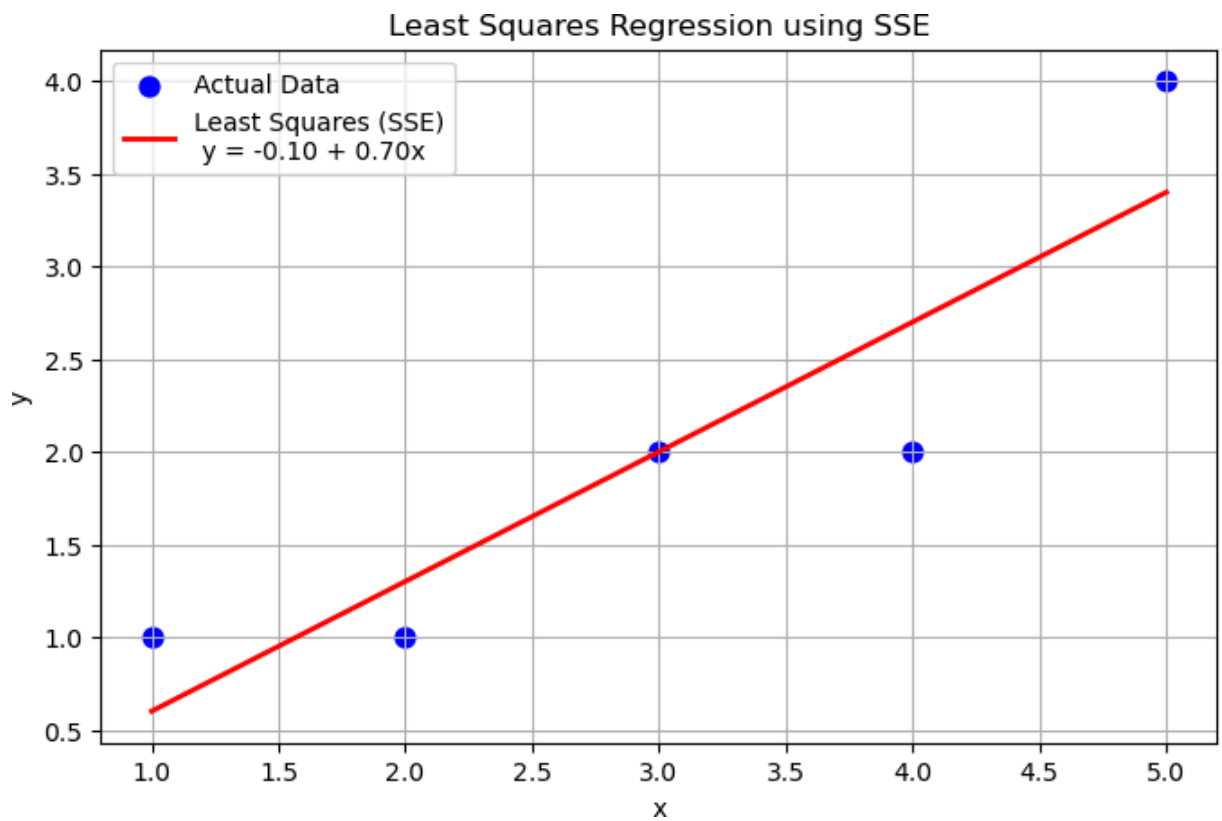
```
Cow Yield Data:

x:[ 4   6 10 12]
y:[3.   5.5 6.5 9. ]

Mean of x: 8.0
 Mean of y: 6.0
b0  (intercept):  0.7999999999999998
b1 (slope): 0.65
y_pred: [3.4 4.7 7.3 8.6]
error: [-0.4  0.8 -0.8      0.4]
```




```
#Birth Weight Data Least Squares Regression Using SSE

x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4]) n =
 len(x)
print(f"Birth Weight Data\n")
 print(f"x = {x}")
print(f"y = {y}") print()

sum_x = np.sum(x)
 sum_y = np.sum(y)
 sum_xy = np.sum(x *
 y)
sum_x2 = np.sum(x ** 2)

print(f"Sum_x = {sum_x}")
 print(f"Sum_y = {sum_y}")
 print(f"Sum_xy = {sum_xy}")
 print(f"Sum_x2 = {sum_x2}")
 print()

a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2) a0 =
 (sum_y / n) - a1 * (sum_x / n)
```

```
print("a0  (intercept):",  a0)
print("a1 (slope):", a1) print()
print(f"y = {a0:.2f} + {a1:.2f}x") print()

y_pred = a0 + a1 * x print(f"y_pred
= {y_pred}") print()

error = (y - y_pred) print(f"error =
{error}") print()

sse = np.sum((y - y_pred) ** 2) print("Sum of Squared
Errors (SSE):", sse)

plt.figure(figsize=(8, 5))

plt.scatter(x, y, color='blue', s=60, label='Actual Data') plt.plot(x, y_pred,
color='red', linewidth=2,
                 label=f'Least Squares (SSE)\n y = {a0:.2f} + {a1:.2f}x')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Squares Regression using SSE') plt.legend()
plt.grid(True)
plt.show()
```

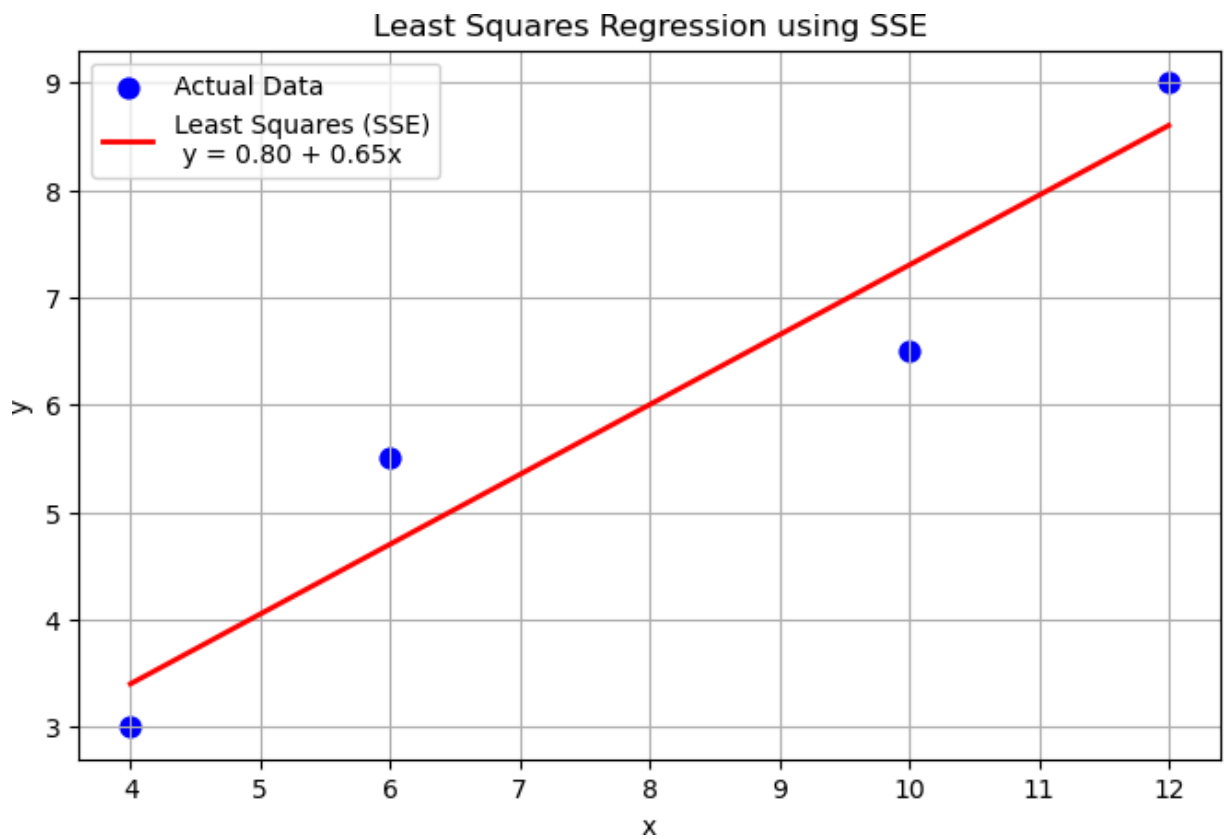
```
Birth Weight Data x =

[1 2 3 4 5]

y = [1 1 2 2 4]

Sum_x = 15
Sum_y = 10
Sum_xy = 37
Sum_x2 = 55

a0  (intercept):  -0.09999999999999964
a1  (slope):  0.7 y =

-0.10 + 0.70x

y_pred = [0.6 1.3 2.                2.7 3.4]

error = [ 0.4 -0.3                0.     -0.7       0.6]

Sum of Squared Errors (SSE): 1.0999999999999999
```




```
#Cow Yield Data Least Squares Regression Using SSE

x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])
n = len(x)
print(f"Cow Yield Data\n")
 print(f"x = {x}")
print(f"y = {y}") print()

sum_x = np.sum(x)
 sum_y = np.sum(y)
 sum_xy = np.sum(x *
 y)
sum_x2 = np.sum(x ** 2)

print(f"Sum_x = {sum_x}")
 print(f"Sum_y = {sum_y}")
 print(f"Sum_xy = {sum_xy}")
 print(f"Sum_x2 = {sum_x2}")
 print()

a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2) a0 =
 (sum_y / n) - a1 * (sum_x / n)
```

```
print("a0  (intercept):",  a0)
print("a1 (slope):", a1) print()
print(f"y = {a0:.2f} + {a1:.2f}x") print()

y_pred = a0 + a1 * x print(f"y_pred
= {y_pred}") print()

error = (y - y_pred) print(f"error =
{error}") print()

sse = np.sum((y - y_pred) ** 2) print("Sum of Squared
Errors (SSE):", sse)

plt.figure(figsize=(8, 5))

plt.scatter(x, y, color='blue', s=60, label='Actual Data') plt.plot(x, y_pred,
color='red', linewidth=2,
                label=f'Least Squares (SSE)\n y = {a0:.2f} + {a1:.2f}x')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Squares Regression using SSE') plt.legend()
plt.grid(True)
plt.show()
```

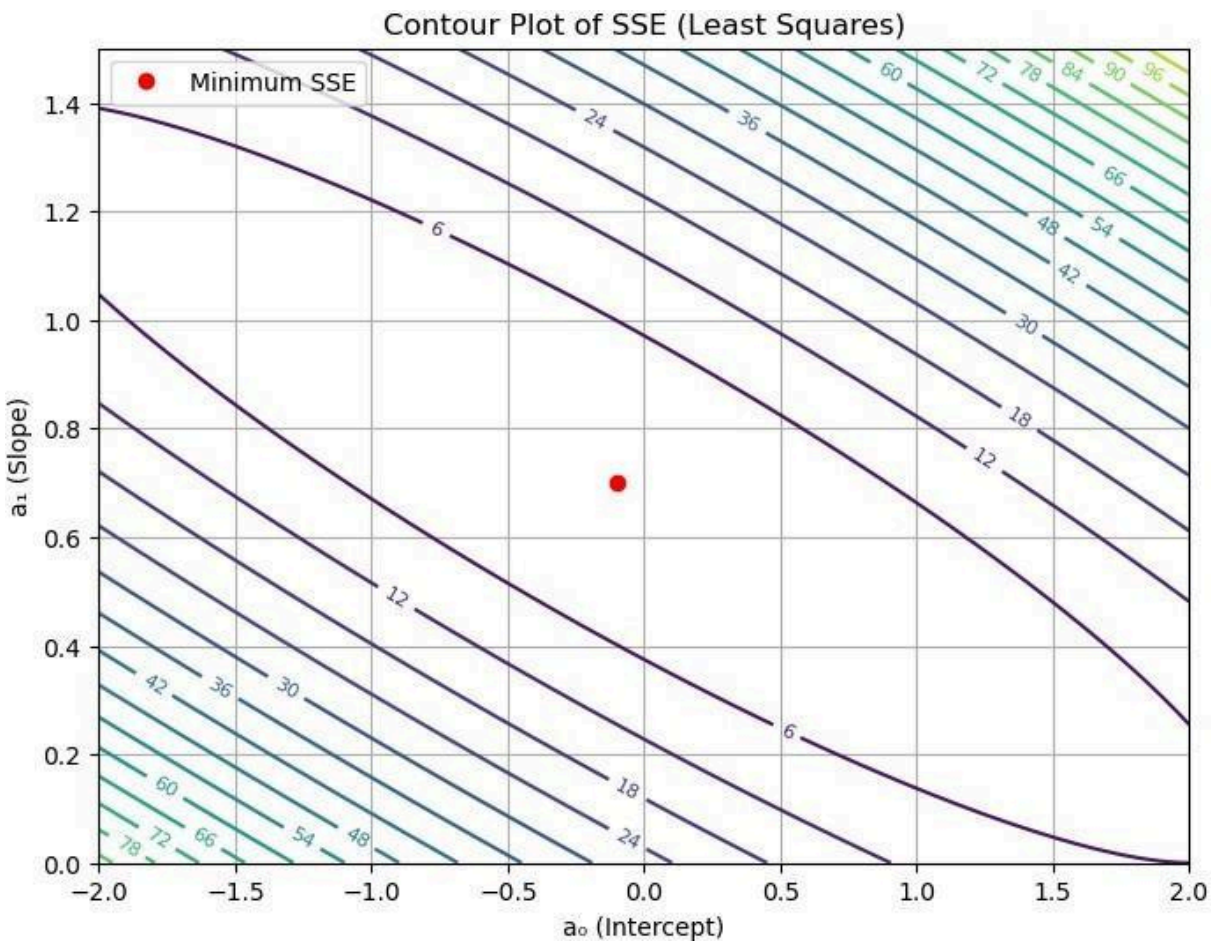
```
Cow Yield Data

x = [ 4        6 10 12]
y = [3.        5.5 6.5 9. ]

Sum_x = 32
Sum_y = 24.0
Sum_xy = 218.0
Sum_x2 = 296

a0  (intercept):  0.7999999999999998
a1 (slope): 0.65 y =

0.80 + 0.65x

y_pred = [3.4 4.7 7.3 8.6]

error = [-0.4          0.8 -0.8        0.4]

Sum of Squared Errors (SSE): 1.599999999999998
```



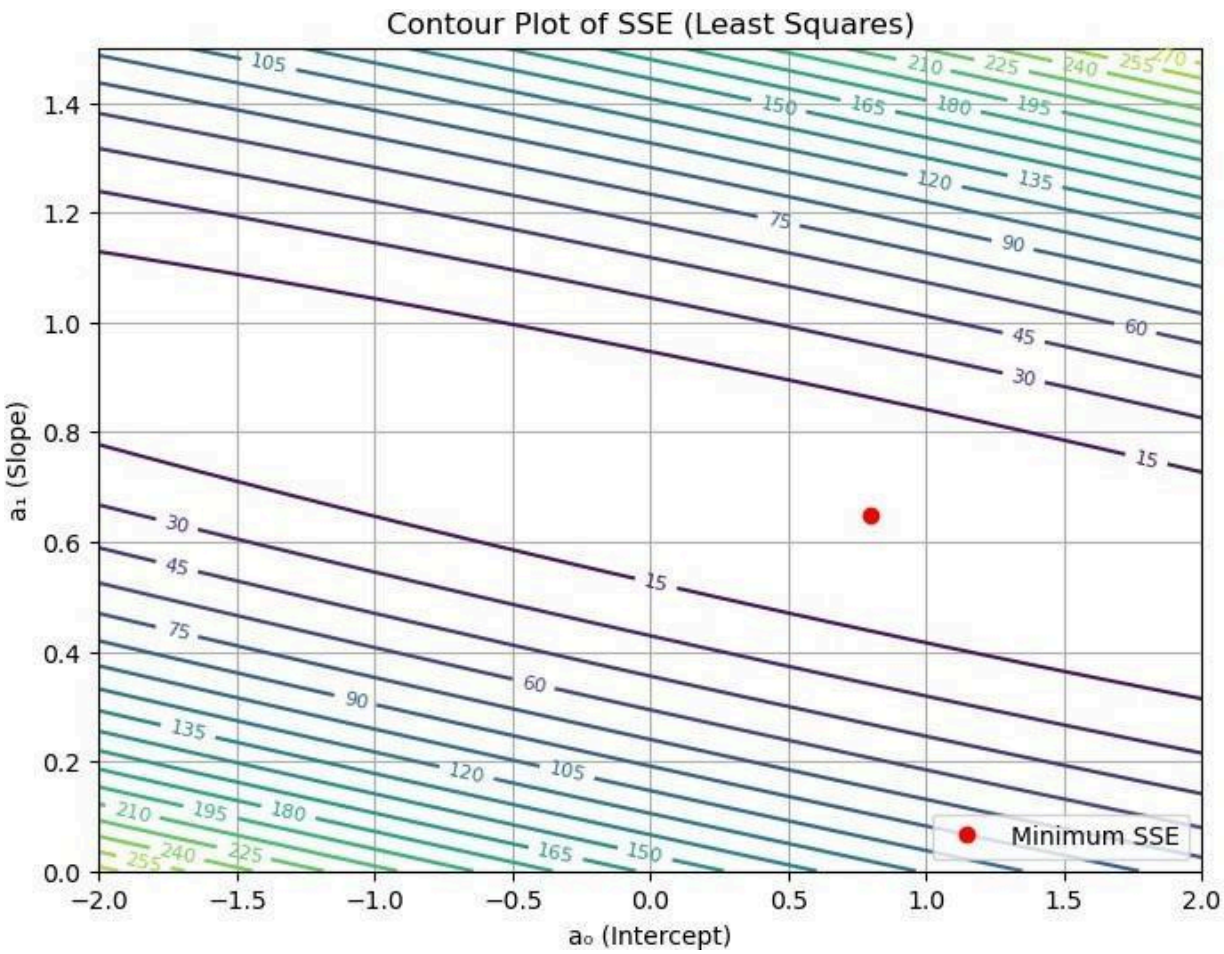

```
#Birth Weight Data
x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4]) n =
 len(x)

# SSE function
def sse(a0, a1):
y_pred = a0 + a1 * x
return np.sum((y - y_pred) ** 2)

a0_vals = np.linspace(-2, 2, 100)
a1_vals = np.linspace(0, 1.5, 100)

A0, A1 = np.meshgrid(a0_vals, a1_vals)
 SSE = np.zeros_like(A0)

for i in range(len(a0_vals)):
for j in range(len(a1_vals)):
SSE[j, i] = sse(A0[j, i], A1[j, i])
```

```
# Least Squares Solution
sum_x = np.sum(x)
 sum_y = np.sum(y)
 sum_xy = np.sum(x *
 y)
```

```
sum_x2 = np.sum(x ** 2)

a1_opt = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
 a0_opt = (sum_y / n) - a1_opt * (sum_x / n)

 # Contour Plot
plt.figure(figsize=(8, 6))

contours = plt.contour(A0, A1, SSE, levels=20, cmap='viridis')
 plt.clabel(contours, inline=True, fontsize=8)

 # Optimal point
plt.plot(a0_opt, a1_opt, 'ro', label='Minimum SSE')

plt.xlabel('a0 (Intercept)')
 plt.ylabel('a1 (Slope)')
plt.title('Contour Plot of SSE (Least Squares)')
 plt.legend()
plt.grid(True) plt.show()
```



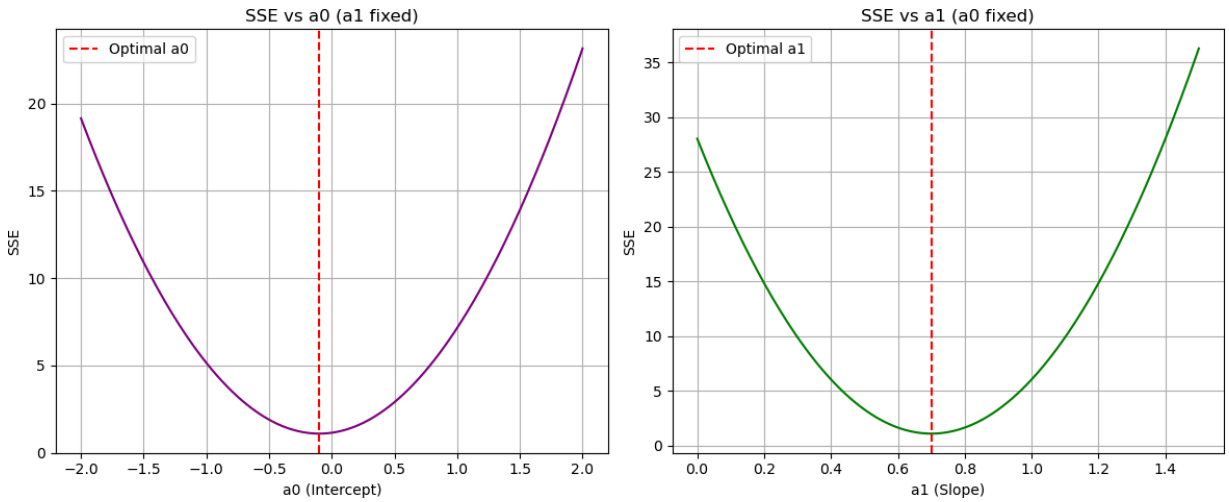
```
#Cow Yield Data
x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])
n = len(x)

# SSE function
def sse(a0, a1):
        y_pred = a0 + a1 * x
        return np.sum((y - y_pred) ** 2)

a0_vals = np.linspace(-2, 2, 100)
a1_vals = np.linspace(0, 1.5, 100)

A0, A1 = np.meshgrid(a0_vals, a1_vals) SSE =
np.zeros_like(A0)

for i in range(len(a0_vals)):
        for j in range(len(a1_vals)):
                SSE[j, i] = sse(A0[j, i], A1[j, i])

# Least Squares Solution
sum_x = np.sum(x) sum_y =
np.sum(y) sum_xy = np.sum(x
* y)
sum_x2 = np.sum(x ** 2)

a1_opt = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2) a0_opt = (sum_y / n) -
a1_opt * (sum_x / n)

# Contour Plot
plt.figure(figsize=(8, 6))

contours = plt.contour(A0, A1, SSE, levels=20, cmap='viridis') plt.clabel(contours, inline=True,
fontsize=8)

# Optimal point
plt.plot(a0_opt, a1_opt, 'ro', label='Minimum SSE')

plt.xlabel('a0 (Intercept)') plt.ylabel('a1 (Slope)')
plt.title('Contour Plot of SSE (Least Squares)') plt.legend()
plt.grid(True)
plt.show()
```

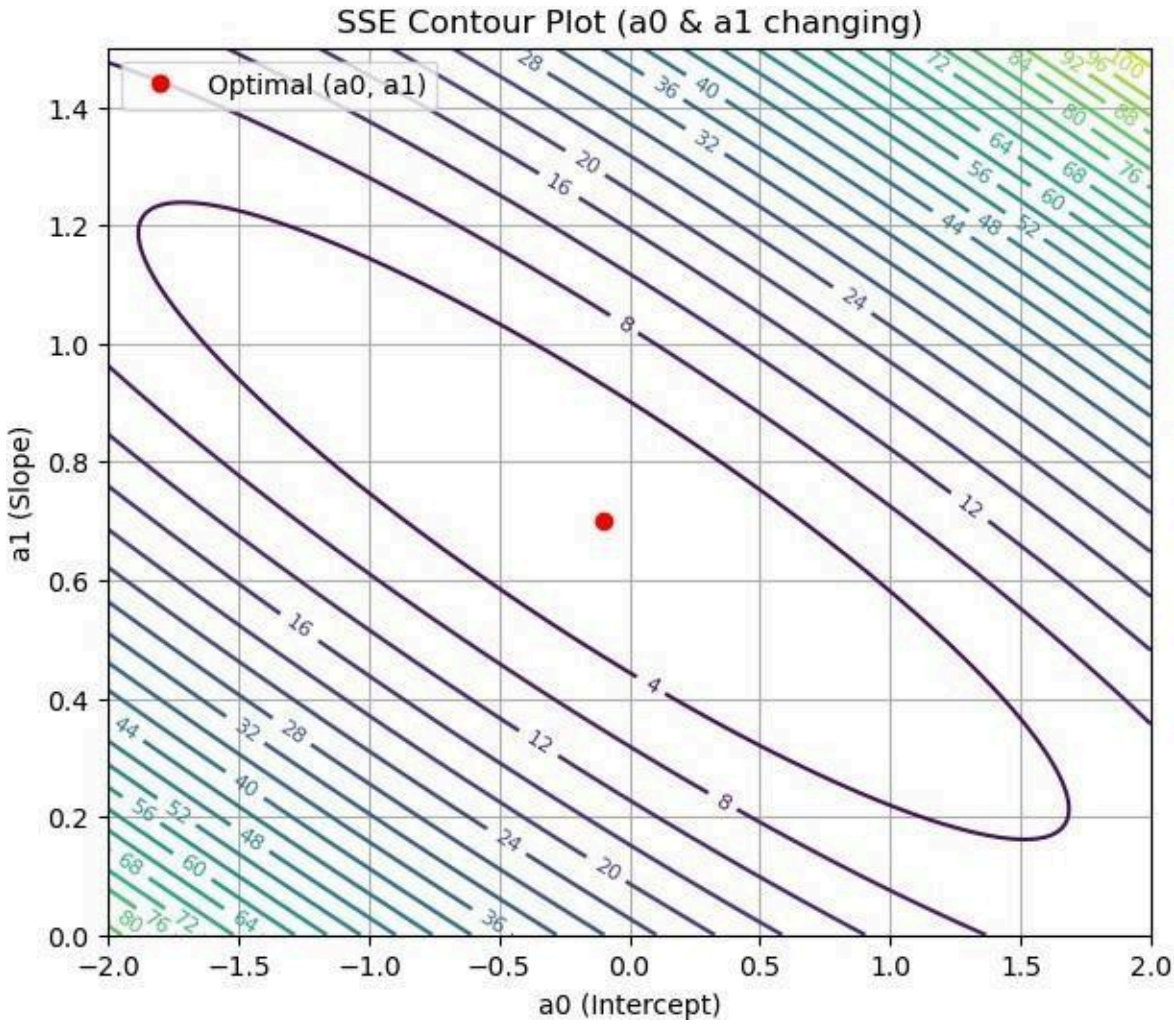


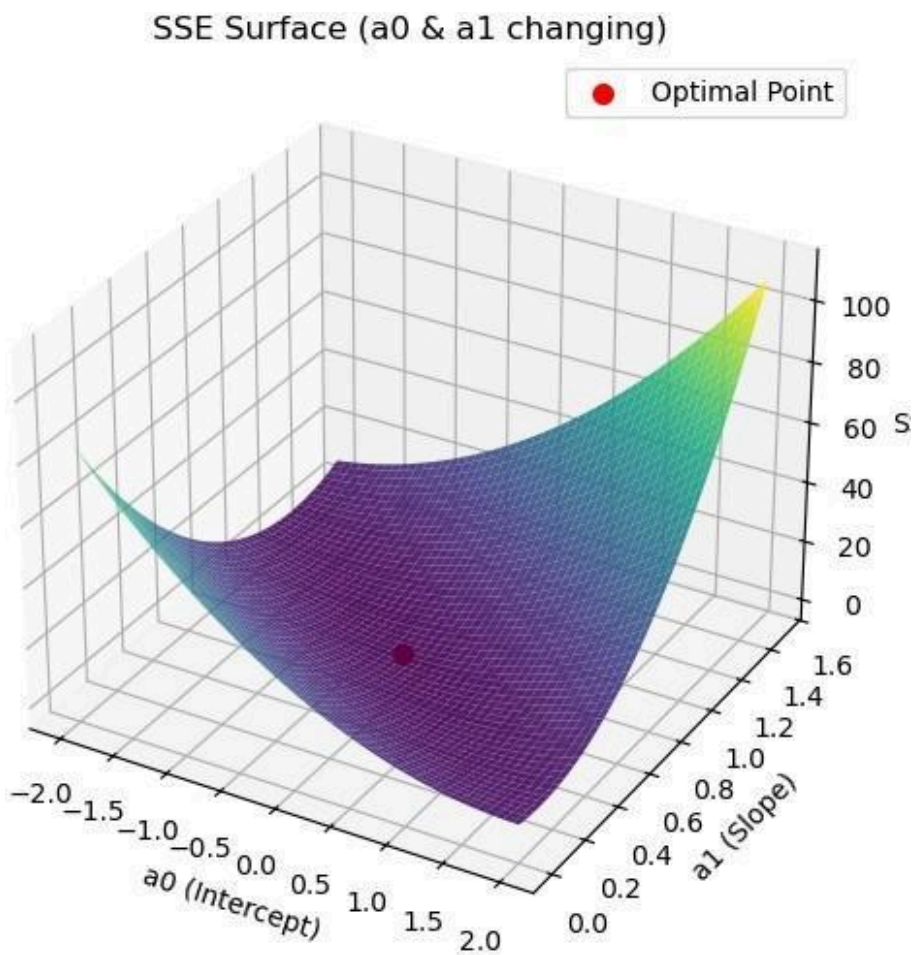

```
#Birth Weight Data
#S(a0,a1) with fixed and changing
x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4]) n =
 len(x)

def sse(a0, a1):
y_hat = a0 + a1 * x
return np.sum((y - y_hat) ** 2)

sum_x = np.sum(x)
 sum_y = np.sum(y)
 sum_xy = np.sum(x *
 y)
sum_x2 = np.sum(x ** 2)

a1_opt = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
 a0_opt = (sum_y / n) - a1_opt * (sum_x / n)

a0_vals = np.linspace(-2, 2, 200)
sse_a0 = [sse(a0, a1_opt) for a0 in a0_vals]
```

```
a1_vals = np.linspace(0, 1.5, 200)
sse_a1 = [sse(a0_opt, a1) for a1 in a1_vals] plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(a0_vals, sse_a0, color='purple')
plt.axvline(a0_opt, color='red', linestyle='--', label='Optimal a0') plt.xlabel('a0 (Intercept)')
plt.ylabel('SSE')
plt.title('SSE vs a0 (a1 fixed)') plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(a1_vals, sse_a1, color='green')
plt.axvline(a1_opt, color='red', linestyle='--', label='Optimal a1') plt.xlabel('a1 (Slope)')
plt.ylabel('SSE')
plt.title('SSE vs a1 (a0 fixed)') plt.legend()
plt.grid(True)

plt.tight_layout() plt.show()

A0, A1 = np.meshgrid( np.linspace(-2,
        2, 200),
        np.linspace(0, 1.5, 200)
)

SSE  =  np.zeros_like(A0)

for i in range(A0.shape[0]):
        for j in range(A0.shape[1]):
                SSE[i, j] = sse(A0[i, j], A1[i, j])

plt.figure(figsize=(7, 6))
contour = plt.contour(A0, A1, SSE, levels=30, cmap='viridis') plt.clabel(contour, inline=True,
fontsize=8)

plt.plot(a0_opt, a1_opt, 'ro', label='Optimal (a0, a1)') plt.xlabel('a0 (Intercept)')
plt.ylabel('a1 (Slope)')
plt.title('SSE Contour Plot (a0 & a1 changing)') plt.legend()
plt.grid(True)
plt.show()
plt.close()
```

```
fig = plt.figure(figsize=(8, 6))
ax  =  fig.add_subplot(111,  projection='3d')

ax.plot_surface(A0, A1, SSE, cmap='viridis', alpha=0.9) ax.scatter(a0_opt,
 a1_opt, sse(a0_opt, a1_opt),
color='red', s=50, label='Optimal Point')

ax.set_xlabel('a0 (Intercept)')
 ax.set_ylabel('a1 (Slope)')
 ax.set_zlabel('SSE')
ax.set_title('SSE Surface (a0 & a1 changing)')
 ax.legend()
plt.show()
```



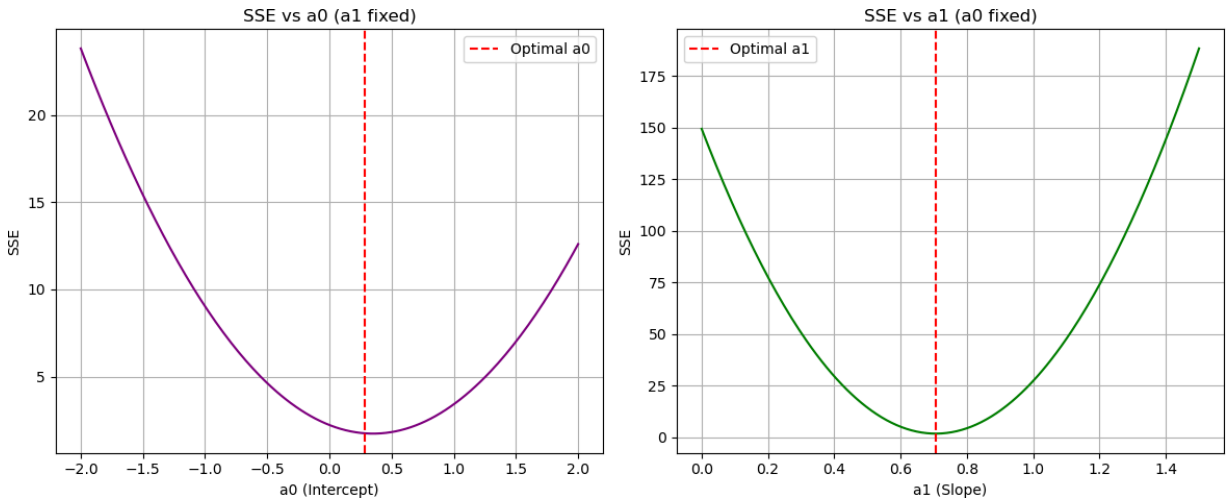

SSE Contour Plot (a0 & a1 changing)
Optimal (a0, a1)
a1 (Slope)
a0 (Intercept)
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
4
8
12
16
20
24
28
32
36
40
44
48
52
56
60
64
68
72
76
80
84
88
92
96
100

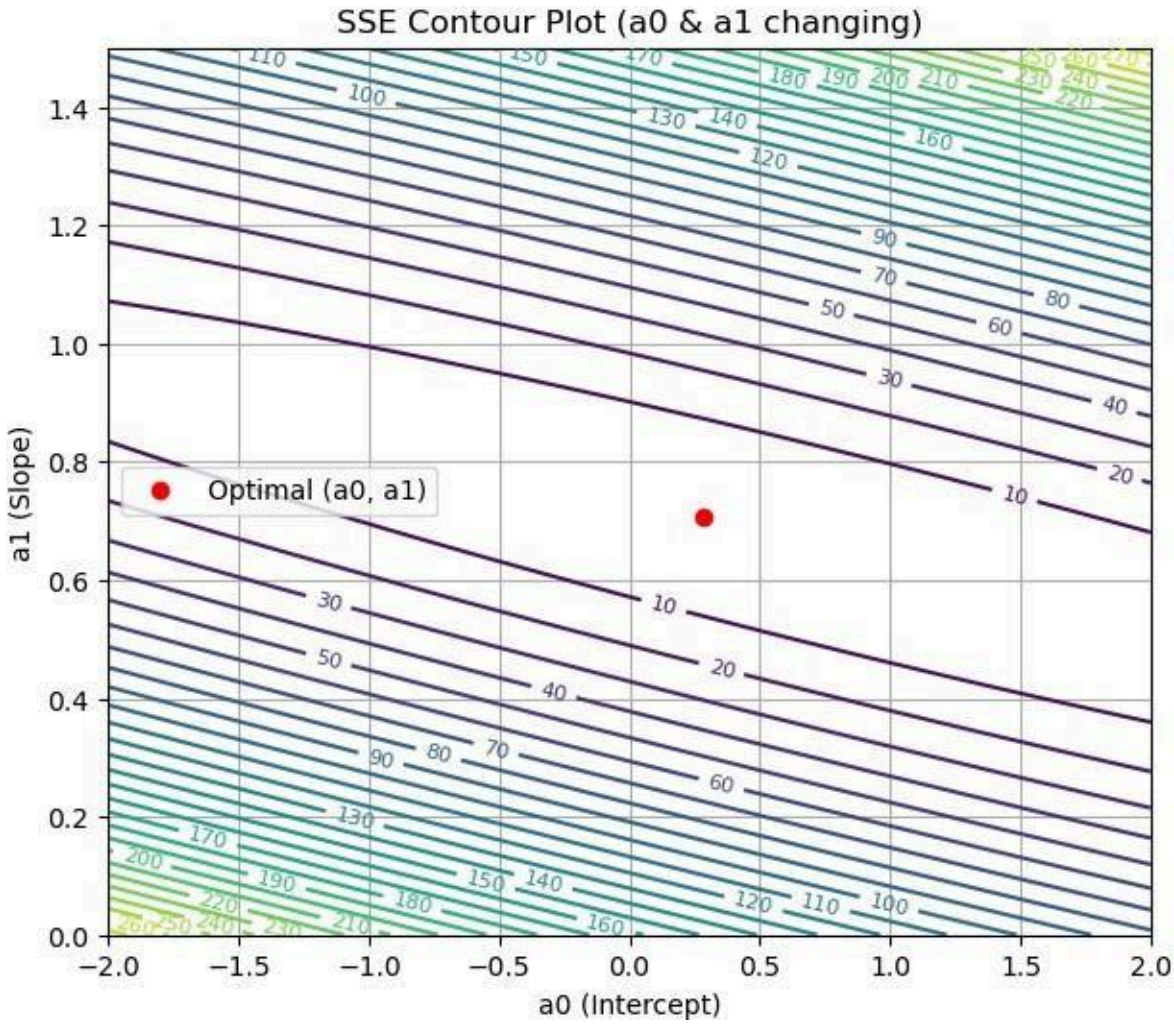


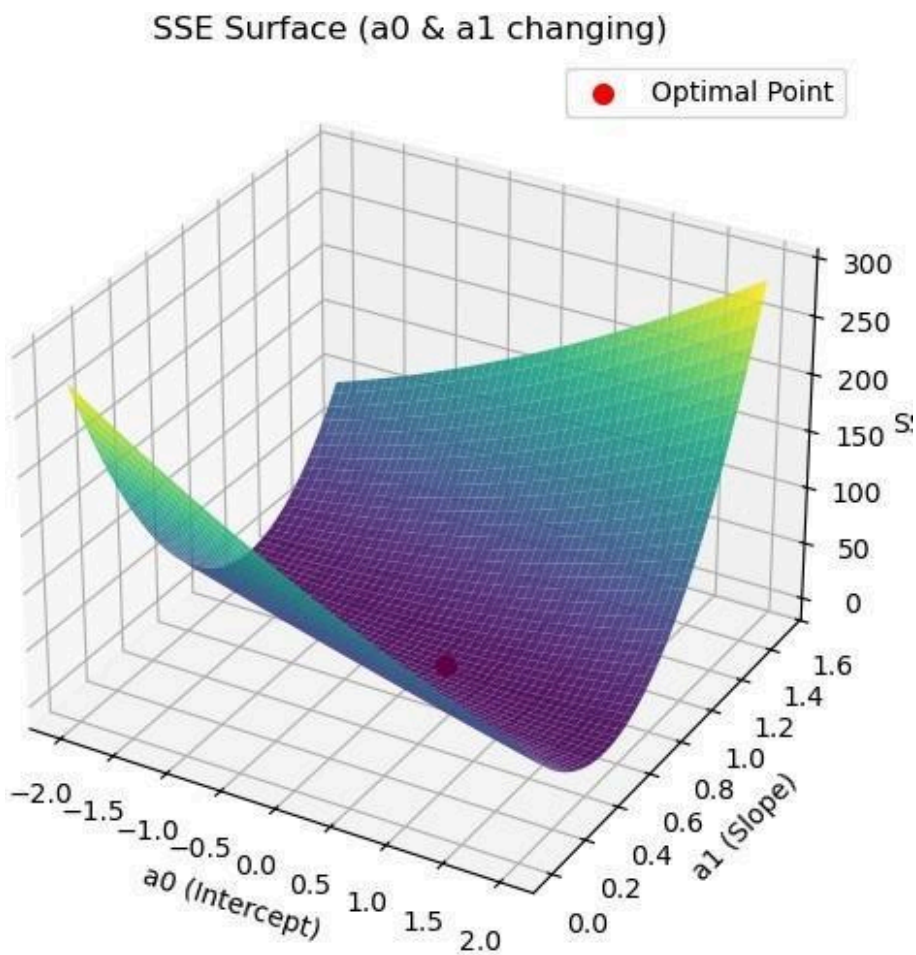

```
#Cow Yield Data
#S(a0,a1) with fixed and changing
x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])

def sse(a0, a1):
y_hat = a0 + a1 * x
return np.sum((y - y_hat) ** 2)

sum_x = np.sum(x)
 sum_y = np.sum(y)
 sum_xy = np.sum(x *
 y)
sum_x2 = np.sum(x ** 2)

a1_opt = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
 a0_opt = (sum_y / n) - a1_opt * (sum_x / n)

a0_vals = np.linspace(-2, 2, 200)
sse_a0 = [sse(a0, a1_opt) for a0 in a0_vals]
```

```
a1_vals = np.linspace(0, 1.5, 200)
sse_a1 = [sse(a0_opt, a1) for a1 in a1_vals] plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(a0_vals, sse_a0, color='purple')
plt.axvline(a0_opt, color='red', linestyle='--', label='Optimal a0') plt.xlabel('a0 (Intercept)')
plt.ylabel('SSE')
plt.title('SSE vs a0 (a1 fixed)') plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(a1_vals, sse_a1, color='green')
plt.axvline(a1_opt, color='red', linestyle='--', label='Optimal a1') plt.xlabel('a1 (Slope)')
plt.ylabel('SSE')
plt.title('SSE vs a1 (a0 fixed)') plt.legend()
plt.grid(True)

plt.tight_layout() plt.show()

A0, A1 = np.meshgrid( np.linspace(-2,
        2, 200),
        np.linspace(0, 1.5, 200)
)

SSE  =  np.zeros_like(A0)

for i in range(A0.shape[0]):
        for j in range(A0.shape[1]):
                SSE[i, j] = sse(A0[i, j], A1[i, j])

plt.figure(figsize=(7, 6))
contour = plt.contour(A0, A1, SSE, levels=30, cmap='viridis') plt.clabel(contour, inline=True,
fontsize=8)

plt.plot(a0_opt, a1_opt, 'ro', label='Optimal (a0, a1)') plt.xlabel('a0 (Intercept)')
plt.ylabel('a1 (Slope)')
plt.title('SSE Contour Plot (a0 & a1 changing)') plt.legend()
plt.grid(True)
plt.show()
plt.close()
```

```
fig = plt.figure(figsize=(8, 6))
ax  =  fig.add_subplot(111,  projection='3d')

ax.plot_surface(A0, A1, SSE, cmap='viridis', alpha=0.9) ax.scatter(a0_opt,
 a1_opt, sse(a0_opt, a1_opt),
color='red', s=50, label='Optimal Point')

ax.set_xlabel('a0 (Intercept)')
 ax.set_ylabel('a1 (Slope)')
 ax.set_zlabel('SSE')
ax.set_title('SSE Surface (a0 & a1 changing)')
 ax.legend()
plt.show()
```



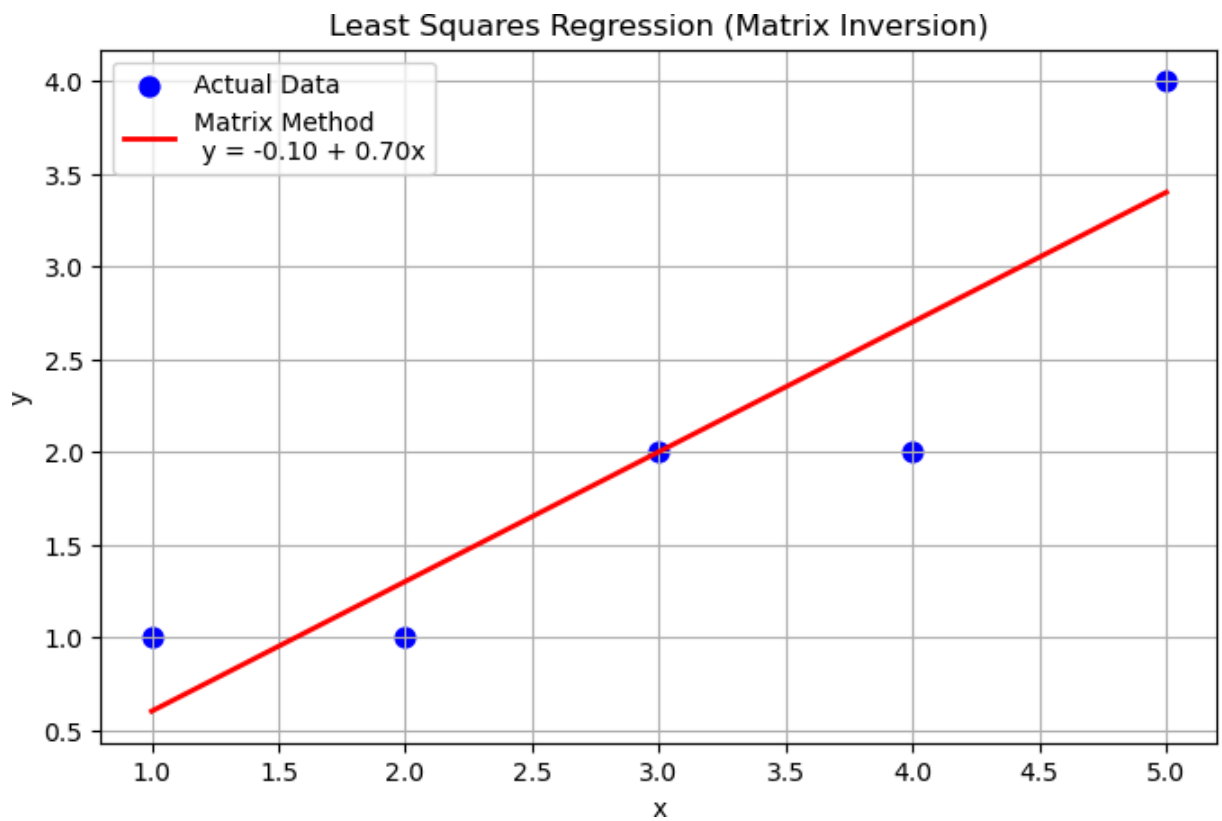

SSE Contour Plot (a0 & a1 changing)
Optimal (a0, a1)
a1 (Slope)
a0 (Intercept)
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0




```
# Birth Weight Data
# Matrix Inversion
x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4])
print(f"x = {x}\ny = {y}\n")

X = np.column_stack((np.ones(len(x)), x)) Y =
 y.reshape(-1, 1)
print(f"X = \n{X}\nY = \n{Y}\n")

# a = (X^T X)^(-1) X^T y
XT = X.T
print(f"XT = \n{XT}")
a = np.linalg.inv(XT @ X) @ XT @ Y print()

a0 = a[0, 0]
a1 = a[1, 0]

print("a0  (intercept):",  a0)
```

```
print("a1  (slope):",  a1)

y_hat = (X @ a).flatten() print(f"y_pred
= {y_hat}") print()

sse = np.sum((y - y_hat) ** 2) print("SSE:", sse)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='blue', s=60, label='Actual Data') plt.plot(x, y_hat, color='red',
linewidth=2,
                label=f'Matrix Method\n y = {a0:.2f} + {a1:.2f}x')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Squares Regression (Matrix Inversion)') plt.legend()
plt.grid(True)
plt.show()
```

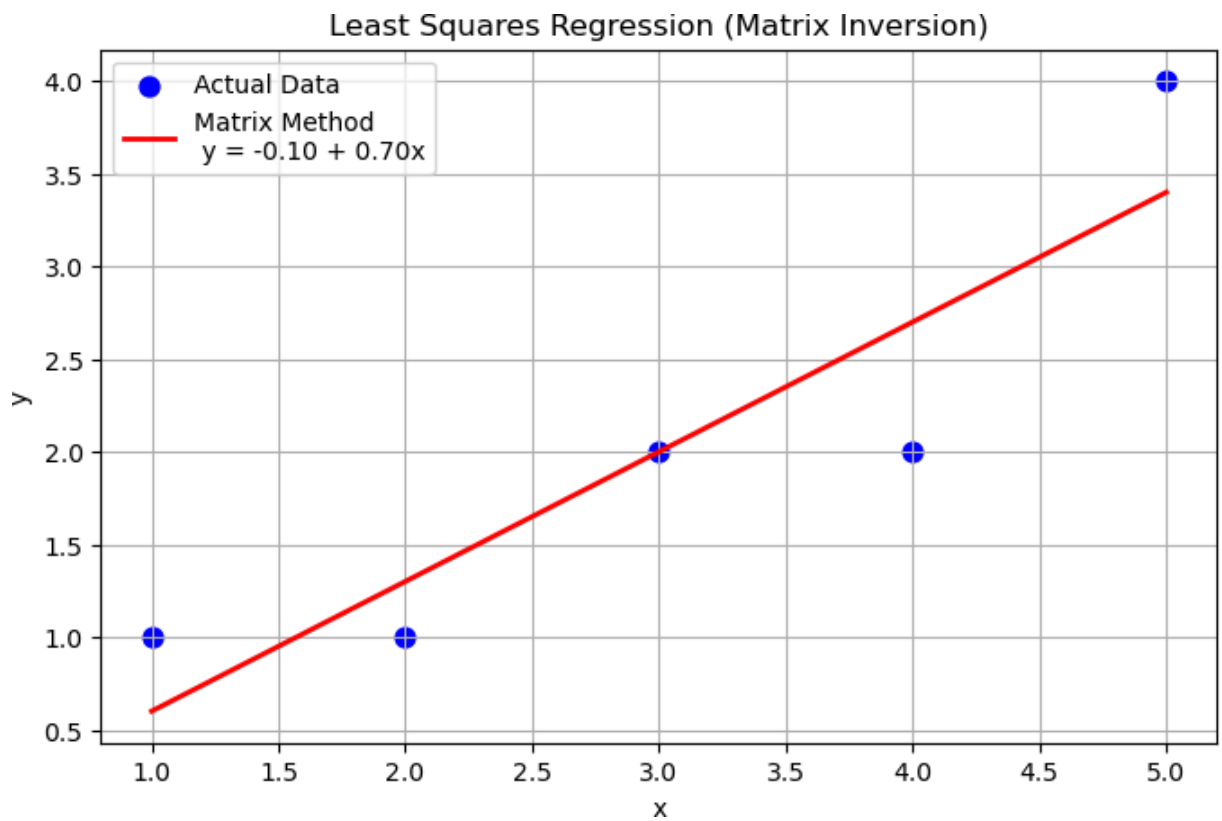
```
x = [1 2 3 4 5]
y = [1 1 2 2 4]

X = [[1.  1.]
 [1. 2.]
 [1. 3.]
 [1. 4.]
 [1. 5.]] Y =
[[1]
 [1]
 [2]
 [2]
 [4]]

XT =
[[1. 1. 1. 1. 1.]
 [1. 2. 3. 4. 5.]]

a0  (intercept):  -0.0999999999999982
a1  (slope):  0.7000000000000005
y_pred = [0.6 1.3 2.              2.7 3.4]

SSE: 1.0999999999999999
```




```
# Birth Weight Data
# Matrix Inversion
x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4])
print(f"x = {x}\ny = {y}\n")

X = np.column_stack((np.ones(len(x)), x)) Y =
 y.reshape(-1, 1)
print(f"X = \n{X}\nY = \n{Y}\n")

# a = (X^T X)^(-1) X^T y
XT = X.T
print(f"XT = \n{XT}")
a = np.linalg.inv(XT @ X) @ XT @ Y print()

a0 = a[0, 0]
a1 = a[1, 0]

print("a0  (intercept):",  a0)
print("a1  (slope):",  a1)

y_hat = (X @ a).flatten()
 print(f"y_pred = {y_hat}")
```

```
print()

sse = np.sum((y - y_hat) ** 2) print("SSE:",

 sse)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='blue', s=60, label='Actual Data') plt.plot(x,
 y_hat, color='red', linewidth=2,
label=f'Matrix Method\n y = {a0:.2f} + {a1:.2f}x')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Squares Regression (Matrix Inversion)') plt.legend()
plt.grid(True) plt.show()
```

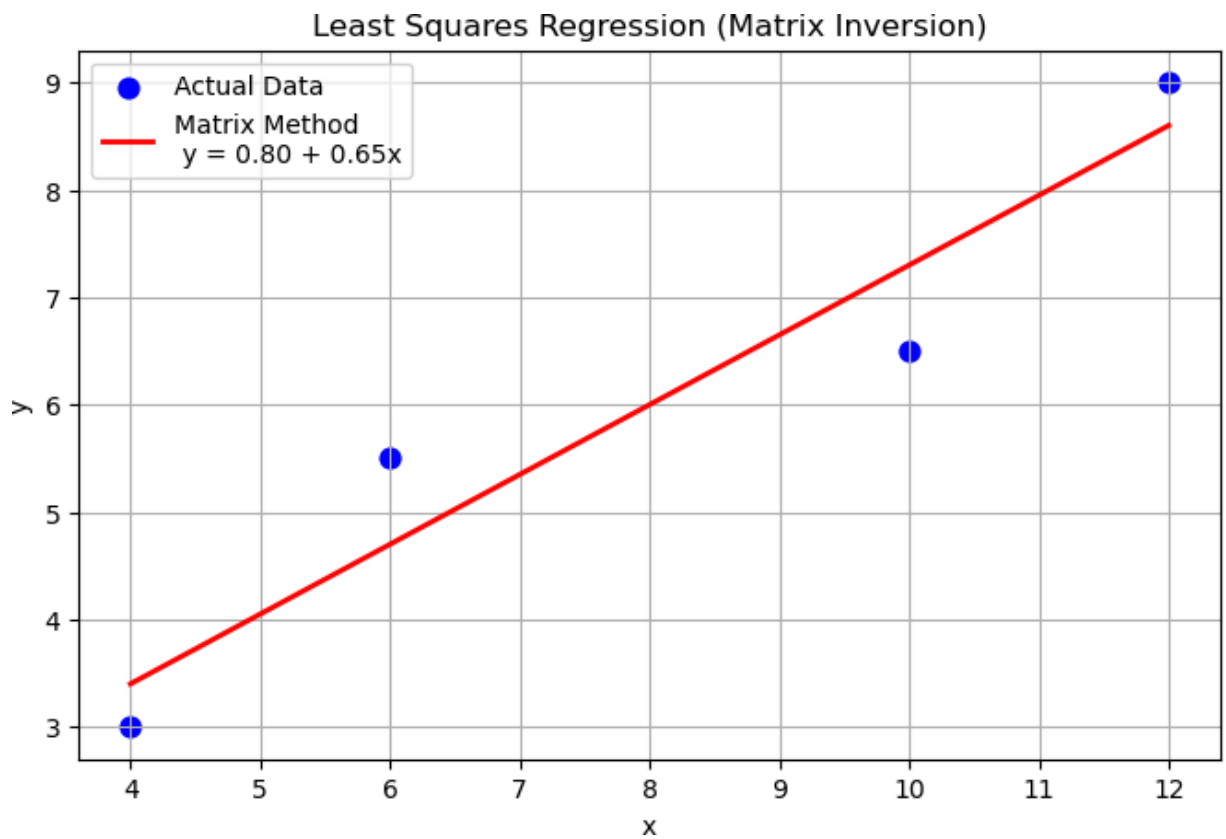
```
x = [1 2 3 4 5]
y = [1 1 2 2 4]

X = [[1.
 1.]
 [1. 2.]
 [1. 3.]
 [1. 4.]
  [1. 5.]] Y
 =
[[1]
 [1]
 [2]
 [2]
 [4]]

XT =
[[1. 1. 1. 1. 1.]
 [1. 2. 3. 4. 5.]]
```




```
# Cow Yield Data
# Matrix Inversion
x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])
print(f"x = {x}\ny = {y}\n")

X = np.column_stack((np.ones(len(x)), x)) Y =
 y.reshape(-1, 1)
print(f"X = \n{X}\nY = \n{Y}\n")

# a = (X^T X)^(-1) X^T y
XT = X.T
print(f"XT = \n{XT}")
a = np.linalg.inv(XT @ X) @ XT @ Y print()

a0 = a[0, 0]
a1 = a[1, 0]

print("a0  (intercept):",  a0)
print("a1  (slope):",  a1)

y_hat = (X @ a).flatten()
 print(f"y_pred = {y_hat}")
```

```
print()

sse = np.sum((y - y_hat) ** 2) print("SSE:",

 sse)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='blue', s=60, label='Actual Data') plt.plot(x,
 y_hat, color='red', linewidth=2,
label=f'Matrix Method\n y = {a0:.2f} + {a1:.2f}x')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Squares Regression (Matrix Inversion)') plt.legend()
plt.grid(True) plt.show()
```

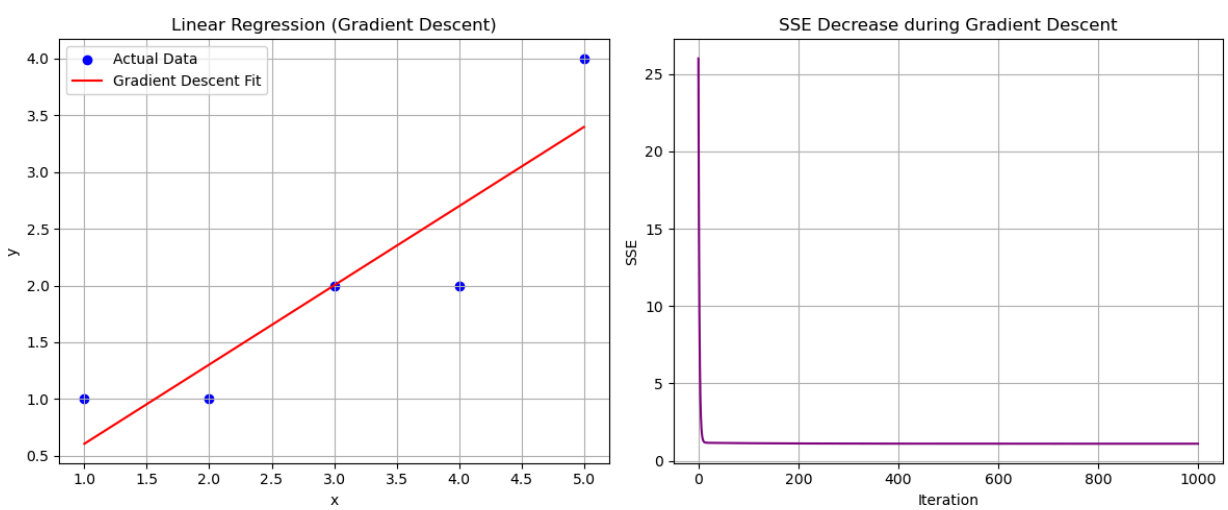
```
x = [ 4        6 10 12]
y = [3.        5.5 6.5 9. ]

X =
[[ 1.   4.]
 [ 1.   6.]
 [ 1. 10.]
  [ 1. 12.]] Y
 =
[[3. ]
 [5.5]
 [6.5]
 [9. ]]

XT =
[[ 1.   1.      1.      1.]
 [ 4.   6. 10. 12.]]
```




```
#Birth Weight Data
#Gradient Descent
x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 1, 2, 2, 4]) n =
 len(x)

b0 = 0.0
b1 = 0.0
alpha = 0.01
epochs = 1000

 cost_history = []

 # Gradient Descent
 for _ in range(epochs):
      y_hat = b0 + b1 * x
      error = y - y_hat

db0 = -2/n * np.sum(error)
db1 = -2/n * np.sum(error * x)

b0 = b0 - alpha * db0 b1 =
      b1 - alpha * db1
```

```
cost = np.sum(error ** 2)
        cost_history.append(cost)

print("Final b0:", b0)
 print("Final b1:", b1)

y_hat = b0 + b1 * x

 plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(x, y, color='blue', label='Actual Data') plt.plot(x, y_hat,
 color='red', label='Gradient Descent Fit') plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear Regression (Gradient Descent)')
 plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2) plt.plot(cost_history,
 color='purple') plt.xlabel('Iteration')
 plt.ylabel('SSE')
plt.title('SSE Decrease during Gradient Descent') plt.grid(True)

plt.tight_layout()
 plt.show()
```

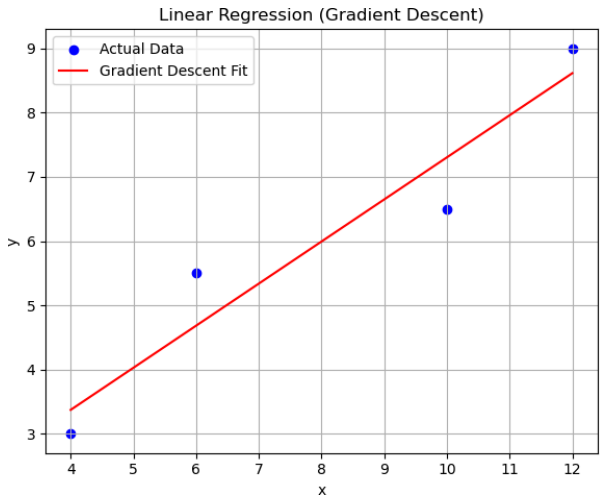
```
Final b0: -0.0907687781409501
Final b1: 0.6974430995681616
```

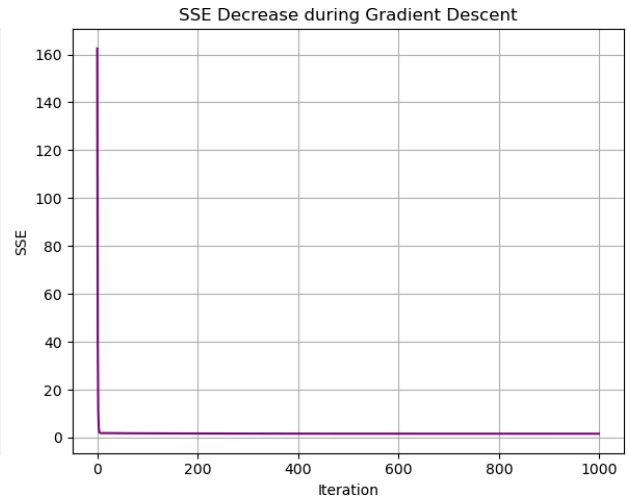



```
#Cow Yield Data
#Gradient Descent
```

```
x = np.array([4, 6, 10, 12])
y = np.array([3.0, 5.5, 6.5, 9.0])
n = len(x)

b0 = 0.0
b1 = 0.0
alpha = 0.01
epochs = 1000

cost_history = []

# Gradient Descent
for _ in range(epochs): y_hat =
        b0 + b1 * x error = y -
        y_hat

        db0 = -2/n * np.sum(error)
        db1 = -2/n * np.sum(error * x)

        b0 = b0 - alpha * db0 b1 =
        b1 - alpha * db1

        cost = np.sum(error ** 2)
        cost_history.append(cost)

print("Final b0:", b0) print("Final
b1:", b1)

y_hat = b0 + b1 * x

plt.figure(figsize=(12,  5))

plt.subplot(1, 2, 1)
plt.scatter(x, y, color='blue', label='Actual Data') plt.plot(x, y_hat, color='red',
label='Gradient Descent Fit') plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear Regression (Gradient Descent)') plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2) plt.plot(cost_history,
color='purple') plt.xlabel('Iteration') plt.ylabel('SSE')
plt.title('SSE Decrease during Gradient Descent') plt.grid(True)

plt.tight_layout() plt.show()
```

```
Final b0: 0.7503082106031975
Final b1: 0.6553817995769352
```





Multiple Linear Regression

Birth Weight Dataset / Cow Yield Dataset

Problem Statement

Implement **Multiple Linear Regression** using the following methods. We consider a dataset with **multiple independent variables**:

$$X = [\, x_1, x_2, x_3, \ldots, x_n \,) \;,\; y = \text{dependent variable}$$

The hypothesis function is:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

---

1. Regular Method (Direct Computation)

Steps

1. Assume the linear model:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

1. Estimate coefficients

$$b_0, b_1, \ldots, b_n$$

manually or iteratively.

2. Compute predicted values:

$$\hat{y}_i = b_0 + \sum_{j=1}^{n} b_j x_{ij}$$

1. Compute error:

$$e_i = y_i - \hat{y}_i$$

1. Evaluate model performance using:
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

---

2. Normal Equations (Least Squares Method)

Model Representation

$$y = X b$$

Where:

- $ X $ is the design matrix with a column of ones
- $ b $ is the parameter vector

Normal Equation

$$b = (X^T X)^{-1} X^T y$$

Cases

- Fixed coefficients $ b_0, b_1, \dots, b_n $
- Simultaneous optimization of all parameters

---

3. Matrix Inversion Method

Step 1: Form the Matrices

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & ¿ & ¿ \end{bmatrix} x_{m1} ¿ x_{m2} ¿ \cdots ¿ x_{mn} ¿$$

$$y = \begin{pmatrix} \\ \vdots \\ y_m \\ y_2 \end{pmatrix}$$

Step 2: Compute

- Determinant of $ X^T X $

- Adjoint of $ X^T X $
- Inverse $ (X^T X)^{-1} $

Step 3: Solve Parameters

$$b=(X^T X)^{-1} X^T y$$

---

4. Gradient Descent Method

Cost Function

$$J(b)=\frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

Partial Derivative

$$\frac{\partial J}{\partial b_j}=\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)\, x_{ij}$$

Update Rule

$$b_j^{\text{new}}=b_j^{\text{old}}-\alpha\frac{\partial J}{\partial b_j}$$

Where:

- $ \alpha $ = learning rate
- $ j = 0, 1, 2, \dots , n $

Algorithm

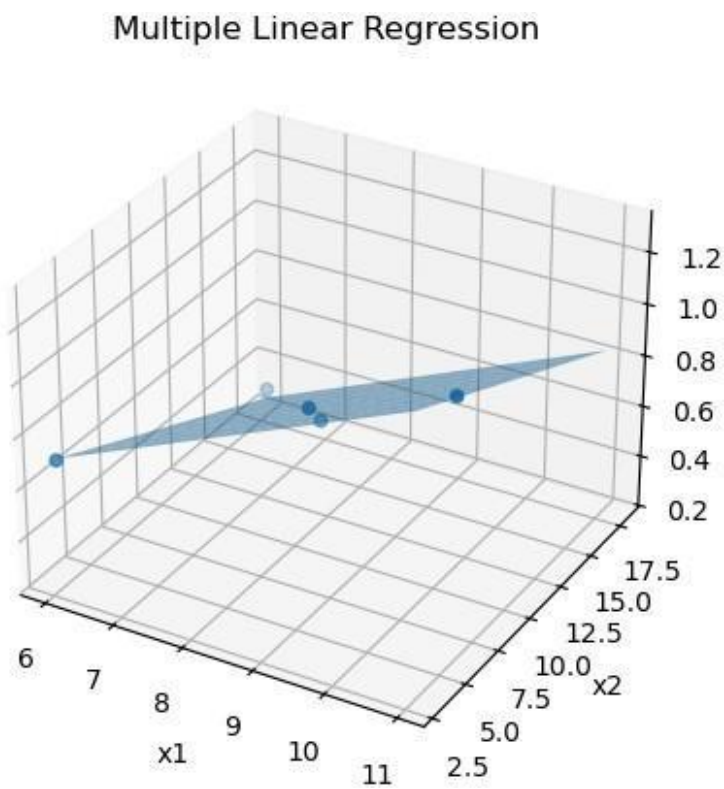
1. Initialize parameters $ b_0, b_1, \dots, b_n $
2. Repeat until convergence:
   - Compute predictions
   - Compute gradients

– Update parameters

3. Plot **Cost vs Iterations**

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.linear_model import SGDRegressor from
sklearn.metrics import mean_squared_error

#Direct Method
x1 = np.array([8, 11, 9, 6, 6], dtype=float)
x2 = np.array([12, 6, 6, 3, 18], dtype=float)
y    = np.array([0.6, 1.2, 1.0, 0.7, 0.3], dtype=float)

X = np.column_stack((np.ones(len(x1)), x1, x2)) print(f"X=\n{X}\n")

B = np.linalg.inv(X.T @ X) @ X.T @ y print(f"B={B}\n")

b0, b1, b2 = B

print("Regression Coefficients:") print("b0 =",
b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = X @ B
print("\nPredicted  values:",  y_pred)

errors = y - y_pred print("errors:",
errors) MSE = np.mean(errors ** 2)
RMSE = np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') ax.scatter(x1, x2, y,

label="Actual Data")

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid
```

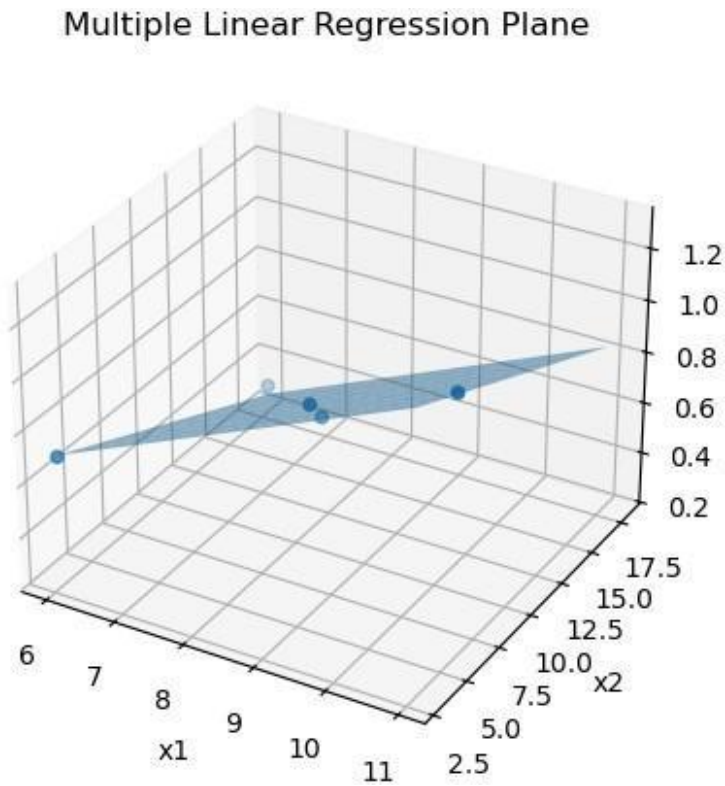
```
ax.plot_surface(x1_grid, x2_grid, y_grid, alpha=0.5)

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression") plt.show()

X=


      [[ 1.      8. 12.]
          [ 1. 11. 6.]
  [ 1.      9.       6.]
  [ 1.      6.       3.]

B=[ 0.10365079        0.11507937 -0.02936508]

Regression Coefficients:
b0 = 0.10365079365079133
b1 = 0.11507936507936496
b2 = -0.029365079365079393

Predicted values: [0.67190476 1.19333333 0.9631746      0.70603175
0.26555556]
errors:  [-0.071904760.00666667     0.0368254       -0.00603175   0.03444444]

Mean Squared Error (MSE): 0.0015587301587301555 Root Mean
Squared Error (RMSE): 0.03948075681556973
```




```
# Normal Equations ( Least Squares Method)
x1 = np.array([8, 11, 9, 6, 6], dtype=float)
x2 = np.array([12, 6, 6, 3, 18], dtype=float)
y      = np.array([0.6, 1.2, 1.0, 0.7, 0.3], dtype=float)

X = np.column_stack((np.ones(len(x1)), x1, x2)) print(f"X
 =  \n{X}\n")

XT = X.T
print(f"XT = \n{XT}\n")

B = np.linalg.inv(XT @ X) @ XT @ y
 print(f"B = {B}\n")

b0, b1, b2 = B

print("Coefficients using Normal Equations:") print("b0
 =", b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = X @ B
 print("\nPredicted y values:")
 print(y_pred)
```

```
errors = y - y_pred

MSE = np.mean(errors ** 2) RMSE =
np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

fig = plt.figure()
ax  =  fig.add_subplot(111,  projection='3d')

ax.scatter(x1, x2, y, label="Actual Data")

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid ax.plot_surface(x1_grid,  x2_grid,  y_grid,

alpha=0.5)

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression Plane") plt.show()

X =
    [[ 1.   8. 12.]
 [ 1. 11.      6.]
 [ 1.    9.     6.]
 [ 1.    6.     3.]
 [ 1.       6. 18.]]

XT =
[[ 1.       1.     1.     1.     1.]
  [ 8. 11.        9.     6.     6.]
 [12.       6.     6.     3. 18.]]

B = [ 0.10365079          0.11507937  -0.02936508]

Coefficients using Normal Equations:
b0 = 0.10365079365079133
b1 = 0.11507936507936496
b2 = -0.029365079365079393

Predicted y values:
[0.67190476 1.19333333 0.9631746              0.70603175 0.26555556]
```

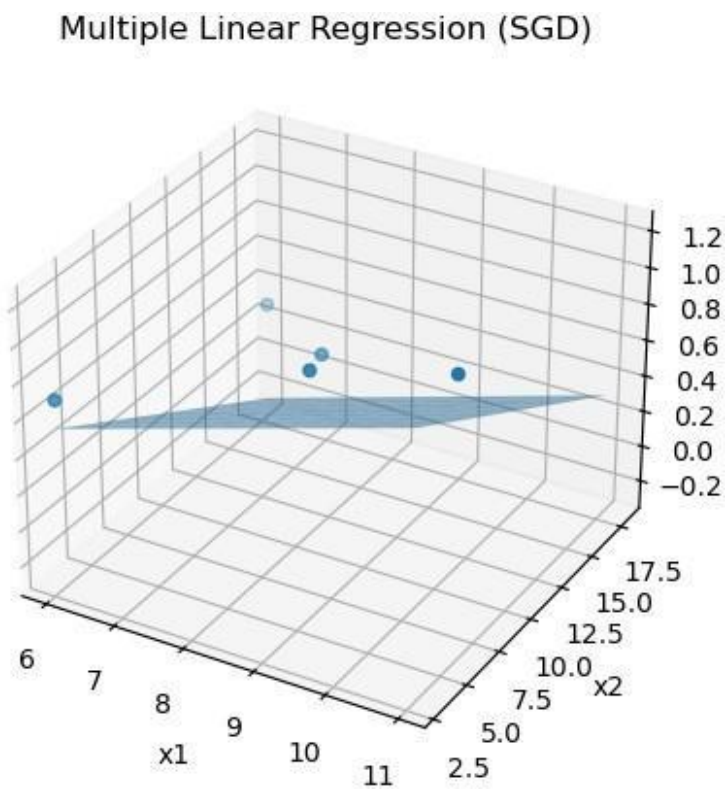
```
Mean Squared Error (MSE): 0.0015587301587301555
 Root Mean Squared Error (RMSE):
 0.03948075681556973
```

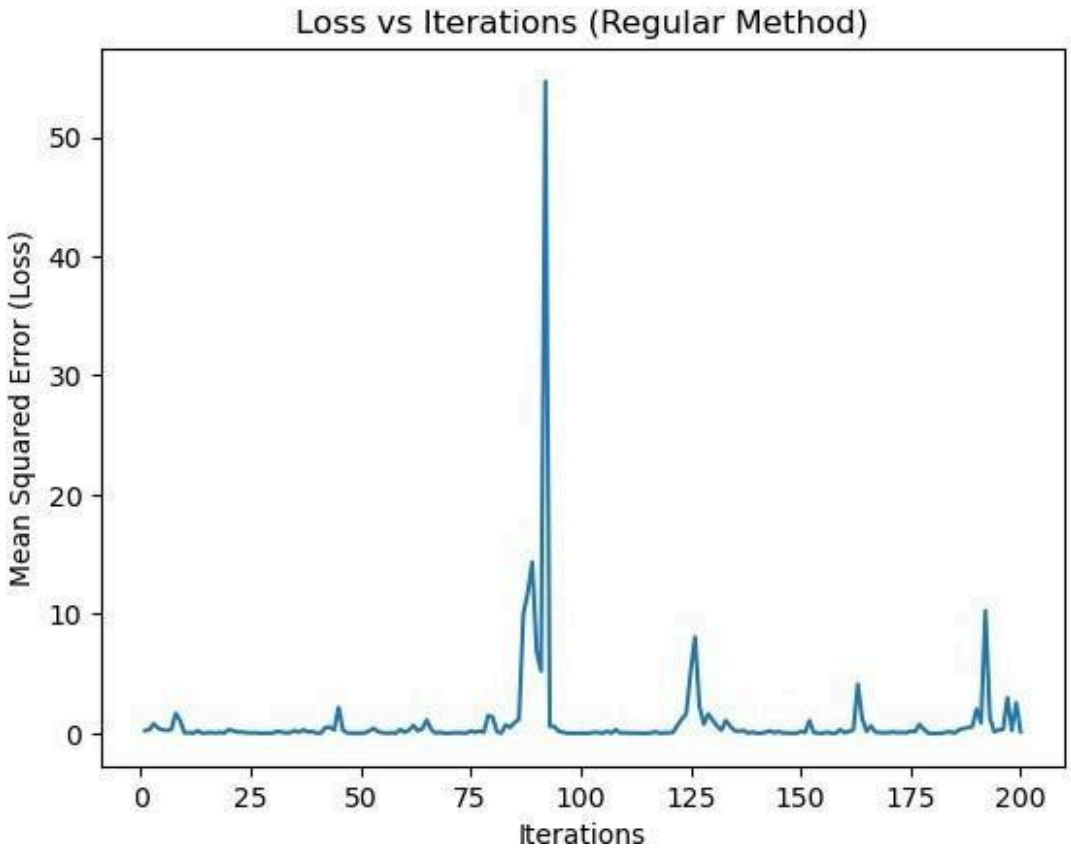



```
 # Direct Method using Libraries
x1 = np.array([8, 11, 9, 6, 6])
x2 = np.array([12, 6, 6, 3, 18])
y       = np.array([0.6, 1.2, 1.0, 0.7, 0.3])

X  =  np.column_stack((x1,  x2))

 model = SGDRegressor(
        learning_rate='constant',
        eta0=0.01,
max_iter=1,
        tol=None,
        warm_start=Tru
        e
 )

iterations = 200 losses
 = []

 for i in range(iterations):
        model.partial_fit(X, y)
        y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
```

```
        losses.append(mse)

b0 = model.intercept_[0] b1, b2 =
model.coef_

print("Regression Coefficients:") print("b0 =",
b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = model.predict(X) errors =
y - y_pred

print("\nPredicted values:", y_pred)
print("Errors:", errors)

MSE = mean_squared_error(y, y_pred) RMSE =
np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') ax.scatter(x1, x2, y)

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid ax.plot_surface(x1_grid,  x2_grid,  y_grid,

alpha=0.5)

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression (SGD)")

plt.show()
plt.close() print()

plt.figure()
plt.plot(range(1, iterations + 1), losses) plt.xlabel("Iterations")
plt.ylabel("Mean Squared Error (Loss)") plt.title("Loss vs Iterations
(Regular Method)") plt.show()
```

```
Regression Coefficients:
b0 = 0.01733428283737028
b1 = 0.11440946195588057
b2 = -0.05309447600416268

Predicted values: [ 0.29547627      0.95727151        0.72845258
 0.54450763 -
0.25190951]
Errors: [0.30452373 0.24272849 0.27154742 0.15549237 0.55190951]

Mean Squared Error (MSE): 0.11083436263549888
Root Mean Squared Error (RMSE): 0.33291795180719663
```



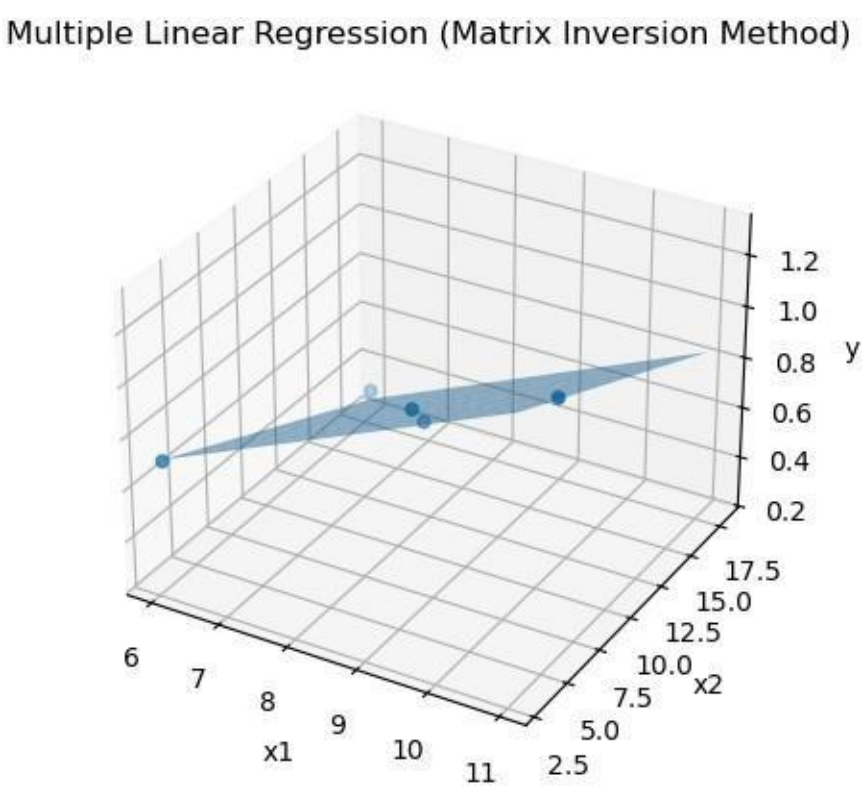



```
# Matrix Inversion Method
x1 = np.array([8, 11, 9, 6, 6], dtype=float)
x2 = np.array([12, 6, 6, 3, 18], dtype=float)
y      = np.array([0.6, 1.2, 1.0, 0.7, 0.3], dtype=float)

X = np.column_stack((np.ones(len(x1)), x1, x2)) print("X
 =\n", X, "\n")

XT = X.T
XTX = XT @ X
print("XᵀX =\n", XTX, "\n")

det_XTX = np.linalg.det(XTX) print("Determinant
 of XᵀX =", det_XTX, "\n")

adj_XTX = np.linalg.inv(XTX) * det_XTX print("Adjoint of
 XᵀX =\n", adj_XTX, "\n")

inv_XTX = np.linalg.inv(XTX) print("Inverse of
 XᵀX =\n", inv_XTX, "\n")

B = inv_XTX @ XT @ y
print("Regression Coefficients Vector B =", B, "\n")
```

```
b0, b1, b2 = B

print("Final Coefficients:") print("b0 =", b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = X @ B
print("\nPredicted  values:",  y_pred)

errors = y - y_pred
MSE = np.mean(errors**2) RMSE =
np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') ax.scatter(x1, x2, y)

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid ax.plot_surface(x1_grid,  x2_grid,  y_grid,

alpha=0.5)

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression (Matrix Inversion Method)") plt.show()
```

```
X =
 [[ 1.          8. 12.]
 [ 1. 11.       6.]
 [ 1.    9.     6.]
 [ 1.    6. 6. 18.]]
XTX =
 [[       5.     40.     45.]
 [ 40. 338. 342.]
 [ 45. 342. 549.]]

Determinant of XTX = 11340.00000000001
```

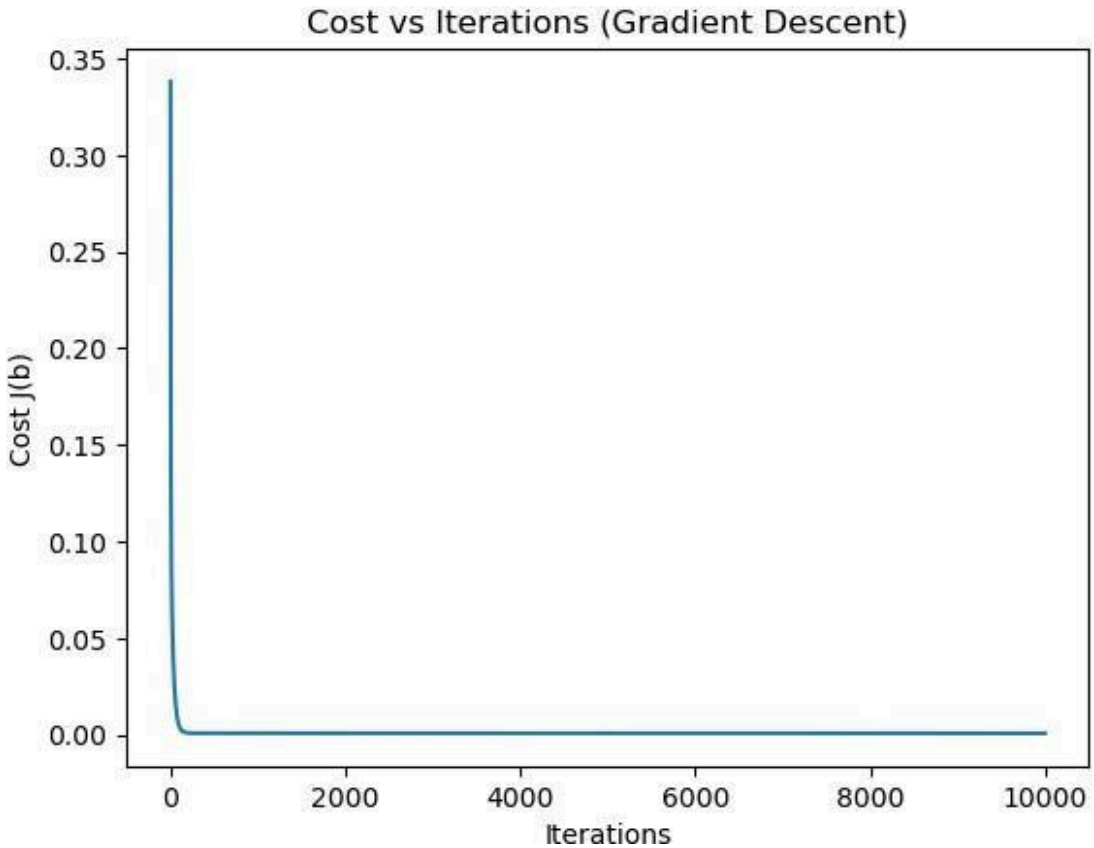
```
Adjoint of XᵀX =
 [[68598. -6570. -1530.]
 [-6570.          720.          90.]
 [-1530.            90.          90.]]

Inverse of XᵀX =
 [[ 6.04920635 -0.57936508 -0.13492063]
 [-0.57936508          0.06349206       0.00793651]
 [-0.13492063          0.00793651       0.00793651]]

Regression Coefficients Vector B = [ 0.10365079                    0.11507937 -
0.02936508]

Final Coefficients:
b0 = 0.10365079365079133
b1 = 0.11507936507936496
b2 = -0.029365079365079393

Predicted values: [0.67190476 1.19333333 0.9631746                    0.70603175
0.26555556]

Mean Squared Error (MSE): 0.0015587301587301555 Root Mean
Squared Error (RMSE): 0.03948075681556973
```



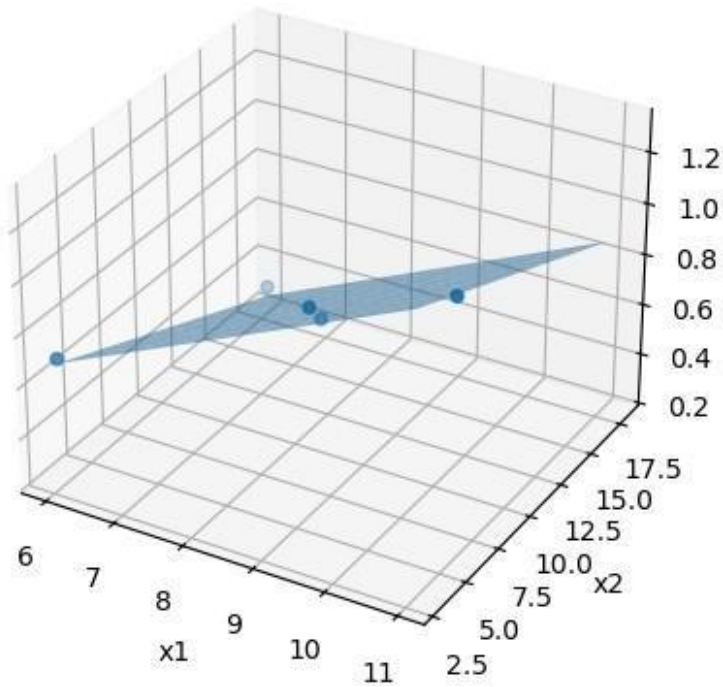
```
# Gradient Descent
x1 = np.array([8, 11, 9, 6, 6], dtype=float)
x2 = np.array([12, 6, 6, 3, 18], dtype=float)
y     = np.array([0.6, 1.2, 1.0, 0.7, 0.3], dtype=float) m = len(y)

b0 = 0.0
b1 = 0.0
b2 = 0.0

alpha = 0.001
iterations = 10000 costs = []

for _ in range(iterations):
        y_hat = b0 + b1 * x1 + b2 * x2

        cost = (1 / (2 * m)) * np.sum((y_hat - y) ** 2) costs.append(cost)

        db0 = (1 / m) * np.sum(y_hat - y)
        db1 = (1 / m) * np.sum((y_hat - y) * x1) db2 = (1 / m) *
        np.sum((y_hat - y) * x2)

        b0 = b0 - alpha * db0 b1 =
        b1 - alpha * db1 b2 = b2 -
        alpha * db2

print("Regression Coefficients (Gradient Descent):") print("b0 =", b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = b0 + b1 * x1 + b2 * x2 errors = y -
y_pred

print("\nPredicted values:", y_pred)
print("Errors:", errors)

MSE = np.mean(errors ** 2) RMSE =
np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

plt.figure() plt.plot(range(iterations), costs)
plt.xlabel("Iterations") plt.ylabel("Cost J(b)")
```

```
plt.title("Cost vs Iterations (Gradient Descent)") plt.show()
plt.close() print()

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') ax.scatter(x1, x2, y)

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid ax.plot_surface(x1_grid,  x2_grid,  y_grid,

alpha=0.5)

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression (Gradient Descent)") plt.show()
```

```
Regression Coefficients (Gradient Descent):
b0 = 0.03707125019090349
b1 = 0.1214632026610084
b2 = -0.027884114928968393

Predicted values: [0.67416749 1.20586179 0.96293538 0.68219812
0.2639364 ]
Errors: [-0.07416749 -0.00586179                    0.03706462       0.01780188       0.0360636 ]

Mean Squared Error (MSE): 0.0017052907058711954 Root Mean
Squared Error (RMSE): 0.04129516564770258
```


Cost vs Iterations (Gradient Descent)
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Cost J(b)
0
2000
4000
6000
8000
10000
Iterations

Multiple Linear Regression (Gradient Descent)



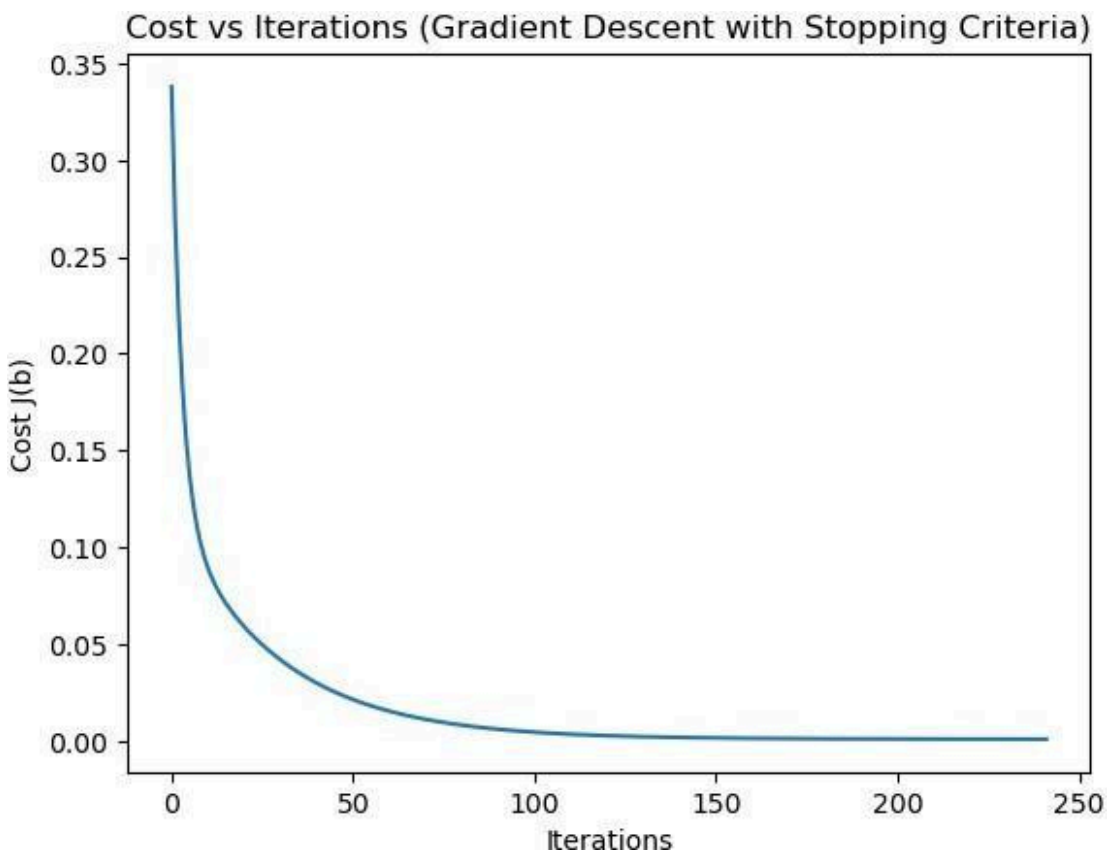
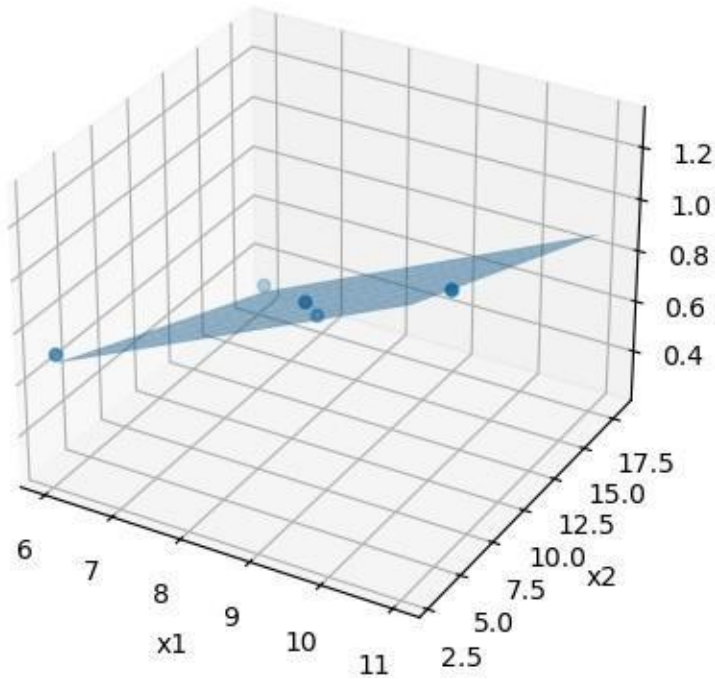

```
# Gradient Descent with stopping criteria
x1 = np.array([8, 11, 9, 6, 6], dtype=float)
x2 = np.array([12, 6, 6, 3, 18], dtype=float)
y      = np.array([0.6, 1.2, 1.0, 0.7, 0.3], dtype=float) m =

len(y)

b0 = 0.0
b1 = 0.0
b2 = 0.0

alpha = 0.001
epsilon = 1e-6
max_iterations  =  10000

costs = []
prev_cost = float("inf")

for i in range(max_iterations): y_hat =
      b0 + b1 * x1 + b2 * x2

cost = (1 / (2 * m)) * np.sum((y_hat - y) ** 2) costs.append(cost)

if abs(prev_cost - cost) < epsilon:
```

```
            print(f"Converged at iteration {i}") break

        prev_cost = cost

        db0 = (1 / m) * np.sum(y_hat - y)
        db1 = (1 / m) * np.sum((y_hat - y) * x1) db2 = (1 / m) *
        np.sum((y_hat - y) * x2)

        b0 -= alpha * db0 b1
        -= alpha * db1 b2 -=
        alpha * db2

print("Regression Coefficients:") print("b0 =",
b0)
print("b1 =", b1)
print("b2 =", b2)

y_pred = b0 + b1 * x1 + b2 * x2 errors = y -
y_pred

MSE = np.mean(errors ** 2) RMSE =
np.sqrt(MSE)

print("\nMean Squared Error (MSE):", MSE) print("Root Mean
Squared Error (RMSE):", RMSE)

plt.figure() plt.plot(costs)
plt.xlabel("Iterations")
plt.ylabel("Cost J(b)")
plt.title("Cost vs Iterations (Gradient Descent with Stopping Criteria)")
plt.show()
plt.close()

print()

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') ax.scatter(x1, x2, y)

x1_range = np.linspace(min(x1), max(x1), 10) x2_range =
np.linspace(min(x2), max(x2), 10) x1_grid, x2_grid =
np.meshgrid(x1_range, x2_range)

y_grid = b0 + b1 * x1_grid + b2 * x2_grid ax.plot_surface(x1_grid, x2_grid, y_grid,

alpha=0.5)
```

```
ax.set_xlabel("x1")
 ax.set_ylabel("x2")
 ax.set_zlabel("y")
ax.set_title("Multiple Linear Regression (Gradient Descent)")

 plt.show()
```

```
Converged at iteration 241
Regression Coefficients:
b0 = 0.011889205371535552
b1 = 0.12244371443721642
b2 = -0.026264388424001948
```

```
Mean Squared Error (MSE): 0.0018908630233407612
 Root Mean Squared Error (RMSE):
 0.043484054817148336
```



Multiple Linear Regression (Gradient Descent)